

密 级： 内部
阶 段： 设计阶段
版 次： V2.0

编译器
Software Requirements Specification
产品型号-BYQ

共 XX 页

测试大学

2026-03-01

编译器

Software Requirements Specification

产品型号-BYQ

编制：_____日期：_____

校对：_____日期：_____

审核：_____日期：_____

标审：_____日期：_____

审定：_____日期：_____

批准：_____日期：_____

目 录

1 范围	1
1.1 标识	1
1.2 系统概述	1
1.3 文档概述	1
2 引用文档	1
3 需求	2
3.1 要求的状态和方式	2
3.2 软件能力需求	2
3.2.X (软件能力)	错误! 未定义书签。
3.3 软件外部接口需求	3
3.3.X (接口的项目唯一的标识符)	错误! 未定义书签。
3.4 软件内部接口需求	4
3.5 软件内部数据需求	5
3.6 适应性需求	5
3.7 安全性需求	5
3.8 保密性需求	6
3.9 软件环境需求	6
3.10 计算机资源需求	6
3.10.1 计算机硬件需求	2
3.10.2 计算机硬件资源使用需求	6
3.10.3 计算机软件需求	6
3.10.4 计算机通信需求	7
3.11 软件质量因素	7
3.12 设计和实现约束	7
3.13 人员需求	7
3.14 培训需求	7

3.15 软件保障需求	7
3.16 其他需求	7
3.17 验收、交付和包装需求（修改有关内容）	8
3.18 需求的优先顺序和关键程度	8
4 合格性规定	8
5 需求可追踪性	8
6 注释	9

1 范围

1.1 标识

本文档适用于编译器软件系统（BYQ），标识号为 BYQ-SRS-V2.0，版本号为 V2.0，发布号为 Release 1。该软件由四川大学网络空间安全学院开发，用于教学和研究目的。

- a) 文档标识号：BYQ-SRS-V2.0;
- b) 标题：Software Requirements Specification
- c) 软件名称：编译器
- d) 软件缩写：BYQ
- e) 软件版本号：V2.0。

1.2 系统概述

本项目是一个用于教学和研究目的的编译器系统，主要功能包括词法分析、语法分析、语义分析、代码生成、代码优化和错误处理。该编译器能够将高级编程语言源代码转换为目标机器的汇编代码或中间代码。

该软件应用于计算机科学教育和编程语言理论研究场景，帮助学生和研究人员深入理解编译原理和技术。开发背景源于四川大学网络空间安全学院的教学需求，旨在提供一个结构清晰、功能完整的编译器实现示例。软件运行在 Windows 和 Linux 操作系统环境下，为计算机专业学生和编程语言研究人员提供服务。

软件的主要特性包括：支持多种源语言特性、生成高效的目标代码、提供详细的错误诊断信息、模块化设计便于扩展和维护。该编译器采用标准的编译流程，包括前端分析（词法、语法、语义分析）和后端生成（代码生成和优化）两个主要阶段。

1.3 文档概述

本文档规定了编译器软件系统的需求规格，包括功能需求、性能需求、接口需求、环境需求和质量需求等方面。文档内容涵盖软件的范围、引用文档、详细需求说明、合格性规定、需求可追踪性和相关注释。

本文档为软件的设计、开发、测试和验收提供依据，适用于软件开发人员、测试人员、项目管理人员和相关技术人员使用。本文档为内部文档，仅限四川大学网络空间安全学院内部使用，不得对外传播。

2 引用文档

Test-引用文档

表 2-1 引用文件

序	名称	版本	发布日期	来源
---	----	----	------	----

号				
1	《GB/T 0018》	0028	2008-12-01	国家质量监督检验检疫总局
2	《GB/T 0028》	0028	2008-12-01	国家质量监督检验检疫总局

3 需求

3.1 要求的状态和方式

本编译器软件在单一状态下运行，不需要区分多种状态或方式。软件在正常工作状态下执行编译任务，包括源代码输入、编译处理和目标代码输出等所有功能。软件不支持维护模式、降级模式或紧急模式等特殊状态。

3.1.1 正常状态

软件在正常状态下运行，执行编译任务，处理用户输入的源代码文件，并生成目标代码文件。软件在正常状态下应满足所有功能需求、性能需求和质量需求。

3.1.2 维护状态

软件在维护状态下运行，进行错误修复、功能改进或性能优化等维护活动。软件在维护状态下可能不满足所有功能需求和性能需求，但应保证基本的稳定性和可靠性。

3.1.3 调试状态

软件在调试状态下运行，开发人员进行调试和测试活动。软件在调试状态下可能不满足所有功能需求和性能需求，但应提供足够的调试信息和日志记录功能，帮助开发人员定位和修复问题。

3.2 软件能力需求

Test 3.2

3.2.1 词法分析模块

功能名称：词法分析模块

功能描述：将源代码文本分解为记号序列，识别语言的基本元素如关键字、

标识符、常量等。

输入：源代码文件

输出：记号序列

性能要求：处理 10000 行以内的源代码，分析时间不超过 5 秒

约束条件：支持 UTF-8 编码的文本文件，需处理注释和空白字符

3.2.2 语法分析模块

功能名称：语法分析模块

功能描述：根据语言的语法规则分析记号序列，构建抽象语法树（AST）。

输入：词法分析模块输出的记号序列

输出：抽象语法树（AST）

性能要求：处理 10000 行以内的源代码，分析时间不超过 5 秒

约束条件：支持递归下降解析，需处理语法错误并提供错误信息

3.2.3 语义分析模块

功能名称：语义分析模块

功能描述：检查程序的语义正确性，包括类型检查、作用域分析和符号表管理。

输入：语法分析模块输出的抽象语法树（AST）

输出：标注后的抽象语法树和符号表

性能要求：处理 10000 行以内的源代码，分析时间不超过 5 秒

约束条件：支持基本类型和用户定义类型，需处理语义错误并提供错误信息

3.3 软件外部接口需求

本编译器软件为独立的离线工具，不需要与外部实体（如网络服务、数据库、远程服务器等）进行接口。软件仅通过文件系统读取源代码文件，并将编译结果输出到文件系统，不涉及外部接口需求。

3.3.1 用户接口

本软件提供命令行界面（CLI）供用户使用，用户通过命令行参数指定源代码文件路径、输出文件路径和编译选项等。软件不提供图形用户界面（GUI）或其他交互方式。

3.3.2 外部系统接口

本软件不需要与任何外部系统进行接口，不涉及网络通信、数据库访问或远程服务调用等外部系统接口需求。

3.4 软件内部接口需求

编译器各模块之间的内部接口需求如下：

1. 词法分析器与语法分析器接口：词法分析器通过记号序列向语法分析器传递识别的词法单元，接口数据结构包括记号类型、记号值、行号和列号等信息。
2. 语法分析器与语义分析器接口：语法分析器将构建的抽象语法树传递给语义分析器，接口数据结构为树形结构，每个节点包含语法成分类型、属性信息和子节点指针。
3. 语义分析器与代码生成器接口：语义分析器将标注后的抽象语法树和符号表传递给代码生成器，接口需包含类型信息、存储分配信息和中间表示。
4. 代码生成器与代码优化器接口：代码生成器将初始目标代码或中间代码传递给优化器，接口需支持控制流图、数据流分析等中间表示。
5. 错误处理模块接口：所有模块通过统一的错误处理接口报告错误，错误信息包括错误级别、错误代码、位置信息和描述信息。

3.4.1 模块间接口规范

编译器内部模块间接口规范如下：

1. 接口定义：每个模块的输入输出数据结构和函数调用约定需明确规定，使用统一的数据格式和命名规范。
2. 错误处理：所有模块通过统一的错误处理机制报告错误，错误信息需包含错误级别、错误代码、位置信息和描述信息。
3. 数据传递：模块间数据传递应采用高效的数据结构，避免不必要的数据复制，支持指针或引用传递。
4. 模块解耦：各模块应尽量解耦，减少直接依赖关系，通过接口抽象实现模块间通信。
5. 接口文档：每个模块的接口需有详细的文档说明，包括输入输出参数、返回值、异常情况和示例。

3.4.2 模块内部接口规范

编译器内部模块内接口规范如下：

1. 函数接口：模块内部函数需遵循统一的命名规范和参数传递方式，函数

应有明确的输入输出定义和返回值。

2. 数据结构接口：模块内部使用的数据结构需有清晰的定义和访问方法，支持封装和抽象，避免直接暴露内部实现细节。

3. 错误处理接口：模块内部函数应通过统一的错误处理机制报告错误，错误信息需包含错误级别、错误代码、位置信息和描述信息。

4. 模块内通信：模块内部组件之间的通信应采用高效的调用方式，支持函数调用、事件通知或消息传递等机制。

5. 内部文档：模块内部接口需有详细的文档说明，包括函数定义、数据结构说明、使用示例和异常情况。

3.5 软件内部数据需求

编译器内部数据结构需求如下：

1. 符号表：用于存储标识符的信息，包括名称、类型、作用域、存储位置等属性。符号表需支持快速查找、插入和删除操作，采用哈希表或平衡二叉树实现。

2. 抽象语法树（AST）：表示源程序的语法结构，每个节点代表一个语法成分。AST 需支持遍历、修改和标注操作，节点类型包括表达式、语句、声明等。

3. 中间代码表示：采用三地址码或静态单赋值形式（SSA）作为中间表示，需支持指令序列的生成、优化和转换。

4. 词法记号流：存储词法分析器产生的记号序列，每个记号包含类型、值、位置等信息。

5. 类型系统：维护语言支持的类型信息，包括基本类型、复合类型和用户自定义类型，需支持类型检查和类型转换。

6. 错误队列：存储编译过程中检测到的所有错误和警告信息，支持按严重程度排序和分类。

3.6 适应性需求

本软件无需特殊的安装配置数据或适应性需求。软件以可执行文件形式提供，用户可直接运行无需复杂配置。

3.7 安全性需求

编译器软件的安全性需求如下：

1. 输入验证：对输入的源代码文件进行合法性检查，包括文件格式、编码方式、文件大小限制等，防止恶意或损坏的输入文件导致编译器崩溃。

2. 错误处理：建立健壮的错误处理机制，当检测到错误时，编译器应能够安全地报告错误并继续运行或优雅地终止，不应因错误导致数据丢失或系统崩溃。

3. 防护措施：防止缓冲区溢出、内存泄漏等编程错误，对内存分配和访问

进行严格检查，避免非法内存访问。

4. 数据安全：编译过程中生成的临时文件和中间结果应妥善管理，编译完成后应清理临时数据，防止敏感信息泄露。

3.8 保密性需求

本软件为教学和研究用途的离线编译器工具，不涉及用户权限管理、数据加密、审计日志或访问控制等保密性需求。软件在本地环境运行，不处理敏感或机密数据。

3.9 软件环境需求

编译器软件的运行环境需求如下：

1. 操作系统：支持 Windows 10 及以上版本、Linux（Ubuntu 18.04 及以上、CentOS 7 及以上）等主流操作系统。
2. 开发工具：需使用 C/C++ 或类似编程语言开发，编译器本身需要 GCC、Clang 或 MSVC 等编译器进行构建。
3. 运行库：依赖标准 C/C++ 运行库，不依赖第三方库或框架。
4. 文件系统：需要读写文件系统的权限，用于读取源代码文件和输出目标代码文件。

3.10 计算机资源需求

3.10.1 计算机硬件需求

处理器：x86-64 架构处理器，主频 1.5GHz 及以上

内存：最低 512MB，推荐 1GB 及以上

存储空间：需要至少 100MB 的磁盘空间用于安装和临时文件存储

3.10.2 计算机硬件资源使用需求

内存使用：编译过程中内存占用不超过 500MB（对于 10000 行以内的源代码）

CPU 使用：编译过程中 CPU 占用率不超过 80%，避免影响其他应用程序

磁盘 I/O：编译过程中读写操作应尽量减少磁盘 I/O 次数，提高编译效率

3.10.3 计算机软件需求

操作系统：Windows 10+ 或 Linux（内核版本 3.0 以上）

运行库：标准 C/C++ 运行时库

构建工具：CMake 或 Make（用于编译器本身的构建）

3.10.4 计算机通信需求

本软件为离线工具，不需要网络通信功能，无计算机通信需求。

3.11 软件质量因素

编译器软件的质量因素要求如下：

1. 正确性：编译器必须正确地实现语言的语义，生成的目标代码必须与源代码的语义一致，通过严格的测试验证。
2. 可靠性：编译器应能够稳定运行，处理各种边界情况和异常输入，错误率不超过 0.1%。
3. 性能：编译速度应满足要求，对于 10000 行以内的源代码，编译时间不超过 10 秒。
4. 可维护性：代码结构清晰，模块化设计，便于后续维护和扩展，代码注释覆盖率不低于 30%。
5. 可移植性：软件应能够在不同的操作系统和硬件平台上编译和运行，遵循标准 C/C++ 规范。
6. 可用性：提供清晰的错误信息和友好的用户界面，便于用户定位和修正程序错误。

3.12 设计和实现约束

本软件无特殊的设计和实现约束，开发团队可根据技术选型和设计决策进行灵活实现。

3.13 人员需求

本软件为教学和研究用途，不需要特殊的人员需求或技能要求。用户需具备基本的编程知识和命令行操作能力。

3.14 培训需求

本软件不需要特殊的培训需求，用户可通过阅读使用手册和帮助文档快速上手。

3.15 软件保障需求

本软件为教学和研究用途，不需要特殊的软件保障需求。软件提供基本的技术支持和文档说明。

3.16 其他需求

无其他特殊需求。

3.17 验收、交付和包装需求（修改有关内容）

软件以安装包形式交付，包含可执行程序、源代码、技术文档和用户手册。

3.18 需求的优先顺序和关键程度

所有功能需求优先级相同，安全性需求和保密性需求为关键需求，需优先满足。

4 合格性规定

Test table

需求名称	需求的唯一标识	本文档中的章节号	合格性方法	合格性级别
《GB/T 0028》	0028	3.3.1	合格性方法	合格性级别
《GB/T 0028》	0028	3.3.1	合格性方法	合格性级别
《GB/T 0028》	0028	3.3.1	合格性方法	合格性级别
《GB/T 0028》	0028	3.3.1	合格性方法	合格性级别
注：合格性方法： a) A-演示：对 CSCI（或部分 CSCI）进行直接可以观察的功能操作，不需要使用复杂或专用的测试设备； b) B-测试； c) C-分析； d) D-检查：对 CSCI 的编码、文档等进行直观检查等； e) E-特殊方法（如：专用工具、技术、过程、设施、验收限制）。 合格性级别： a) a-配置项：在 CSCI 中发现的质量问题； b) b-系统集成：在系统集成时发现的问题； c) c-系统：在系统中发现的质量问题； d) d-系统安装：在系统安装时发现的质量问题。				

5 需求可追踪性

Test table

表 5-1 需求的正向追踪性

《XXX 课题可行性研究报告》中的指标		软件需求	
指标内容	章节号	需求名称	章节号
指标内容	章节号	需求名称	章节号
指标内容	章节号	需求名称	章节号
指标内容	章节号	需求名称	章节号
若存在一对多则此形式表达			

表 5-2 需求的逆向追踪性

软件需求		《XXXXX 课题可行性研究报告》中的指标	
需求名称	指标内容	指标内容	章节号
需求名称	指标内容	指标内容	章节号
需求名称	指标内容	指标内容	章节号
需求名称	指标内容	指标内容	章节号

6 注释

术语定义：

编译器（Compiler）：将高级编程语言源代码转换为目标代码的软件工具。

词法分析（Lexical Analysis）：将源代码字符流分解为记号序列的过程。

语法分析（Syntax Analysis）：根据语法规则分析记号序列，构建抽象语法树的过程。

语义分析（Semantic Analysis）：检查程序的语义正确性，包括类型检查和作用域分析。

抽象语法树（Abstract Syntax Tree, AST）：源程序的树形表示，每个节点代表一个语法成分。

符号表（Symbol Table）：存储标识符信息的数据结构。

中间代码（Intermediate Code）：源代码与目标代码之间的中间表示形式。

代码优化（Code Optimization）：改进代码性能的过程，不改变程序语义。

缩略语：

AST - Abstract Syntax Tree（抽象语法树）

SRS - Software Requirements Specification（软件需求规格说明）

BYQ - 编译器（Bian Yi Qi）

SSA - Static Single Assignment (静态单赋值)